Day - 6: Interrupts (Part 2)

▼ External Interrupts | Edge Trigger | Level Trigger

1. Introduction to External Interrupts

- **Definition:** An external interrupt is a signal sent to the microcontroller from an external source, prompting it to halt its current operation to handle the request.
- **Sources:** Unlike regular internal interrupts (which come from peripherals like SPI, I2C, or timers), external interrupts are generated by external devices such as physical push buttons or sensors experiencing a signal change (from 0 to 1, or 1 to 0).
- **Dedicated Pins:** Each external interrupt is specifically assigned to a dedicated port pin on the microcontroller (for example, External Interrupt 0 might be assigned to pin 2.0, External Interrupt 1 to pin 2.1, etc.).

2. Configuration Modes & Registers

External interrupts can be configured into four different combinations based on how they trigger. This is controlled using two primary registers: **EXTMODE** and **EXTPOLAR**.

- **Edge-Triggered Mode:** Triggers on a signal transition.
  - Setting `EXTMODE = 1` configures the interrupt as edge-triggered.
  - **Rising Edge:** Triggers when the signal goes from low to high. Set `EXTPOLAR = 1`.
  - **Falling Edge:** Triggers when the signal goes from high to low. Set `EXTPOLAR = 0`.
- **Level-Triggered Mode:** Triggers based on the continuous state of the signal.

- Can be configured as either **Active High** or **Active Low** using the same two registers.

3. Coding and Configuration Steps

To properly implement an external interrupt in code, several steps must be followed:

1. **Pin Configuration:** Assign a specific microcontroller pin to act as the external interrupt (e.g., port pin P2.12).
2. **Mode Configuration:** Write to the `EXTMODE` and `EXTPOLAR` registers to define the trigger behavior (e.g., setting both to `1` for an edge-triggered, rising-edge interrupt).
3. **Enable NVIC:** The interrupt must be enabled on the hardware side using the `enable IRQ` functionality from the NVIC (Nested Vectored Interrupt Controller).
4. **Set Priority:** Assign a priority level to the interrupt. Depending on the system, you can assign up to 7 different priority levels (e.g., setting the external interrupt priority to 1).

4. Execution Flow: Main Code & ISR

- **Main Code Execution:** The main program initializes the clocks, pins, and interrupts, and then enters a continuous `while(1)` loop. During normal operation, it simply runs this loop (e.g., printing "CPU is normal").
- **Hardware Trigger:** When the physical switch is pressed or the sensor state changes, an interrupt signal is generated.
- **Interrupt Service Routine (IRQ Handler):** The microcontroller immediately stops the main loop and jumps to the IRQ Handler function.
- **Clearing the Flag:** Inside the handler, the code executes its required tasks (like printing "external interrupted") and **must clear the interrupt flag** so the system knows the request was handled. After this, the processor returns to normal operation.

▼ NVIC API Functions for Interrupts | CMSIS APIs

1. Introduction to CMSIS

- **Definition:** CMSIS stands for **Common Microcontroller Software Interface Standard**.
- **Origin & Purpose:** It is an open-source software standard created by ARM to standardize coding across all ARM and ARM Cortex-based microcontrollers.
- **Main Goals:**
  - **Software Reusability:** Allows developers to reuse the same code across different microcontrollers without having to rewrite it from scratch.
  - **Time Efficiency:** Significantly decreases the time required to develop code and reduces the learning curve for new microcontrollers.
- **Components:** The standard includes defined software components, APIs (function calls), standard rules, tools, and workflow designs.

2. NVIC Core Functionalities under CMSIS

The **NVIC** (Nested Vectored Interrupt Controller) falls directly under the CMSIS standard. CMSIS defines a set of standardized API functions for the NVIC to handle interrupt programming smoothly across different devices.

3. Key NVIC API Functions

To manage interrupts, developers use these standard CMSIS-defined APIs, which require the specific interrupt's IRQ number as a parameter:

- **NVIC_EnableIRQ(IRQ_number):** Enables the specified interrupt.
- **NVIC_DisableIRQ(IRQ_number):** Disables the specified interrupt.
- **NVIC_GetPendingIRQ(IRQ_number):** Checks if a specific interrupt is currently in a pending state waiting to be serviced.
- **NVIC_SetPendingIRQ(IRQ_number):** Manually sets a specific interrupt into a pending state.
- **NVIC_ClearPendingIRQ(IRQ_number):** Clears the pending state of an interrupt.

- **NVIC_GetActive(IRQ_number):** Checks whether a specific interrupt is currently active or not.
- **NVIC_SetPriority(IRQ_number, priority_level):** Sets the priority of a specific interrupt. It takes two parameters: the IRQ number and the desired priority level (up to 80 different priorities can be set depending on the microcontroller).
- **NVIC_GetPriority(IRQ_number):** Returns the priority level at which a specific interrupt is currently running.

4. IRQ Numbers and Header Files

- **Finding IRQ Numbers:** To use the NVIC APIs, you must pass the specific IRQ (Interrupt Request) number. These numbers are not guessed; they are predefined in the microcontroller's specific **header file** (e.g., the LPC17xx header file) or the data sheet.
- **Enumerated Types (enum):** System developers use `enum` data types in the header files to map specific peripheral interrupts to their exact numbers.
    - *Example:* Exceptions like Non-Maskable Interrupts might be negative numbers (e.g., -14), while peripheral interrupts start from 0 (e.g., Watchdog Timer = 0, Standard Timer = 1).
- **Coding Best Practice:** Instead of remembering and passing raw numbers (like `0` or `1`), it is highly recommended to pass the defined interrupt names (e.g., `TIMER0_IRQn`) into the API functions for easier and more readable code development.

▼ Tail Chaining in Interrupts | Late Arrival | Pop Pre-emption

1. Introduction to Tail Chaining

- **The Problem:** In embedded systems handling multiple interrupts, the processor normally has to continuously perform **stacking** (saving the context/state) and **unstacking** (restoring the context) for each interrupt. This continuous storing and restoring consumes significant time and increases interrupt latency.

- **The Solution:** Tail Chaining is a hardware-based optimization mechanism used in ARM and other embedded systems to improve interrupt handling efficiency.
- **Advantages:** It significantly reduces execution time, decreases latency, ensures faster Interrupt Service Routine (ISR) execution, and increases overall energy efficiency.

2. The Core Mechanism

- Tail chaining specifically activates when there are multiple, back-to-back interrupts (e.g., one executing and one pending).
- Instead of saving and restoring the context for the second interrupt, the processor entirely **skips the intermediate unstacking and stacking processes**.
- After the first interrupt finishes, the processor directly jumps into the ISR of the second interrupt, effectively creating a continuous "chain" of executions without wasting time saving state information in between.

3. Three Tail Chaining Scenarios

The video breaks down the tail chaining concept into three distinct operating scenarios based on when interrupts arrive and their priorities:

Scenario A: Standard Tail Chaining

- **Condition:** Two interrupts request service simultaneously. IRQ1 has a higher priority, and IRQ2 has a lower priority.
- **Execution Flow:**
    1. Because IRQ1 is higher, the processor performs stacking for IRQ1 and executes ISR1, while IRQ2 is kept in a pending state.
    2. When ISR1 finishes, instead of performing the normal unstacking of IRQ1 and stacking of IRQ2, the processor skips these steps.
    3. It directly jumps to execute ISR2.
    4. Only after ISR2 is fully completed does the processor perform a single unstacking process.

Scenario B: Late Arrival

- **Condition:** IRQ2 (lower priority) occurs first. However, *during* the initial stacking process of IRQ2, IRQ1 (higher priority) arrives.
- **Execution Flow:**
  1. Even though IRQ2 arrived first, the system always prioritizes the higher priority interrupt.
  2. Because IRQ1 arrives while IRQ2 is still stacking, IRQ1 is immediately served first.
  3. After ISR1 executes, the processor utilizes the tail chain concept to jump straight to ISR2, followed by the final unstacking process.

Scenario C: Pop Pre-emption

- **Condition:** IRQ1 (higher priority) executes completely. However, *during* the unstacking (popping) process of IRQ1, IRQ2 suddenly arrives.
- **Execution Flow:**
  1. The processor immediately halts (suspends) the unstacking/popping process.
  2. Rather than finishing the unstacking and starting over, it directly jumps to execute ISR2.
  3. Once ISR2 is completed, the suspended unstacking process is finally allowed to finish. This specific skipping mechanism is known as **stack pop pre-emption**.

▼ Interrupt Pre-emption | Nested Interrupts | Context Switching

1. Introduction to Multiple Interrupts

- **Context:** A microcontroller contains various built-in peripherals (such as ADCs, timers, UART, and SPI), and each of these peripherals is capable of generating its own independent interrupt.
- **The Challenge:** When a system utilizes multiple peripherals simultaneously, it must be properly configured to handle multiple nested interrupts based on their importance.

2. Structuring the Code for Multiple Interrupts

To successfully manage multiple interrupts (for example, using both a Timer interrupt and an External interrupt), the software must be broken down into distinct parts:

- **Configuration Functions:** You must write a separate configuration function for every interrupt source you intend to use (e.g., `interrupt_configuration_1` for the timer and `interrupt_configuration_2` for the external pin).
- **Interrupt Service Routines (ISRs):** Similarly, you must define dedicated handler functions for each source. If the timer generates an interrupt, the processor jumps to `ISR1`; if the external source generates one, it jumps to `ISR2`.

3. Priority Assignment & Rules

- **Enabling & Prioritizing:** During the configuration phase, each interrupt must be explicitly enabled and assigned a specific priority level.
- **The Priority Rule:** In this architecture, a **lower number dictates a higher priority**, while a higher number dictates a lower priority.
- **Example Setup:** If you configure a Timer interrupt with a priority of `1` and an External interrupt with a priority of `2`, the Timer will possess the higher priority.

4. Main Execution & Clearing Flags

- **Main Code Loop:** The main program initializes the necessary configuration functions and then enters an infinite `while(1)` loop (e.g., performing a normal task like printing "normal"). It will continuously run this loop until an interrupt triggers (like the timer elapsing every 5 seconds or a physical switch being pressed).
- **Clearing the Flag:** Inside each ISR, once the requested task is executed, it is **mandatory to clear the interrupt flag** before exiting. If this flag is not cleared, the processor will not properly recognize that the interrupt has been handled.

5. Context Switching & Interrupt Pre-emption

- **The Pre-emption Mechanism:** The true benefit of assigning priorities is seen when interrupts overlap. If a lower-priority interrupt (like the External interrupt at priority 2) is actively executing its ISR, and a higher-priority interrupt (like the Timer at priority 1) suddenly triggers, the processor will perform a **context switch**.
- **Execution Flow:** The system will automatically pause (suspend) the lower-priority external interrupt, override it to immediately execute the higher-priority timer interrupt, and only return to finish the external interrupt once the timer ISR is complete.